

Autoencoders

Sistemas de Inteligencia Artificial

23 de junio de 2026

ITBA — 2026

Autoencoder basico

El problema y el dataset font.h

Dataset: 32 caracteres de una fuente bitmap.

- Cada letra: grilla $7 \times 5 = 35$ **pixeles** binarios $\{0, 1\}$.

Pregunta gancho: ¿se pueden comprimir 35 pixeles a **2 numeros** y reconstruirlos con ≤ 1 **pixel** de error?

Objetivo

Aprender los **32 patrones** de 5×7 en un latente de **2 dimensiones** con un error maximo de **1 pixel incorrecto**.

Ejemplo (letra 'a'):

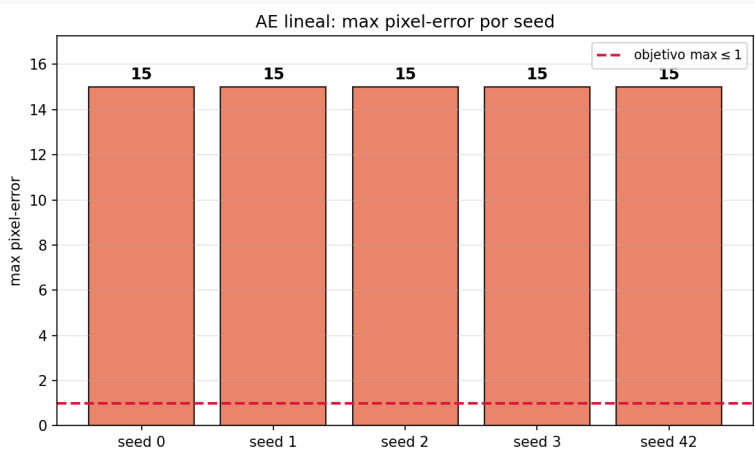
```
.....  
....#  
.####  
#...#  
#...#  
.##.#  
.....
```

Primer intento — ¿alcanza con un AE lineal?: parametros

Parametro	Valor
Datos	font.h (32 × 35)
Arquitectura	[35, 60, 40, 20, 2, 20, 40, 60, 35]
Activaciones	todas identidad (lineal)
Perdida	MSE
Optimizador	Adam, $lr = 5 \cdot 10^{-3}$
Adam $\beta_1, \beta_2, \epsilon$	0.9, 0.999, 10^{-8}
Batch	full-batch (32)
Init	Xavier
Epocas	20000
Seeds	[0, 1, 2, 3, 42]
Metrica	max pixel-error + rec-MSE

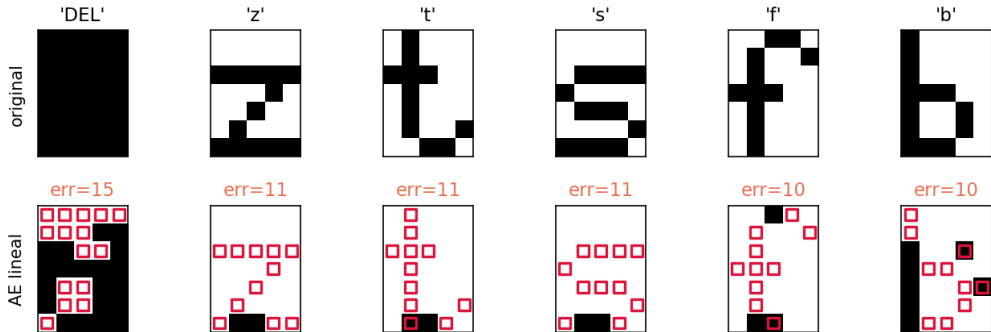
Pregunta: ¿alcanza un autoencoder *lineal* (la version mas simple) para reconstruir las 32 letras con ≤ 1 pixel de error?

Primer intento — ¿alcanza con un AE lineal?: resultados

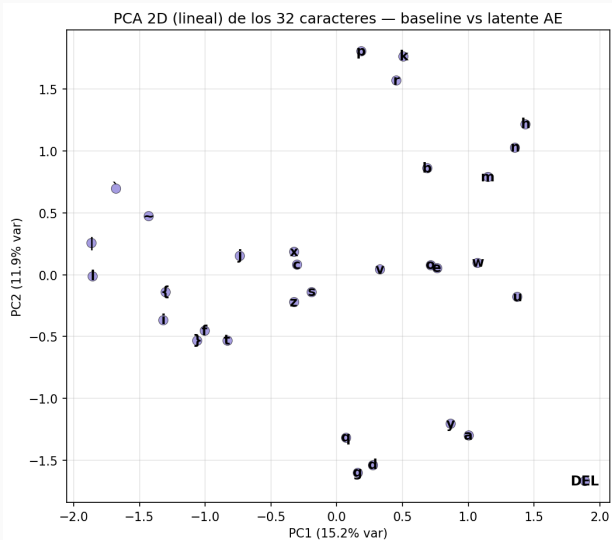


Una letra concreta: 'DEL' pierde 15 de 35 pixeles

AE lineal: peores reconstrucciones (rojo = pixel erroneo)



¿Por que falla? Converge a PCA



- Un AE *lineal* converge a PCA.
- Su latente 2D retiene solo **~27 %** de la varianza $\Rightarrow$ no separa las 32 letras.

La respuesta: no linealidad — arquitectura base del AE

Misma idea simetrica con **cuello 2D**, pero **ocultas no lineales**:

$35 \rightarrow 60 \rightarrow 40 \rightarrow 20 \rightarrow 2 \rightarrow 20 \rightarrow 40 \rightarrow 60 \rightarrow 35$

- Latente **linear** (no satura); salida **logistica** (pixeles en $(0, 1)$).

Optimizadores

- GD
- Momentum
- Adam

Funciones de perdida

- BCE
- MSE

Activaciones ocultas

- tanh
- ReLU
- logistica

Esta es la *configuracion base* de referencia: los experimentos varian **una** pieza a la vez.

Estudio de optimizacion

Parametro	Valor
Datos	font.h (32×35)
Config base	[35, 60, 40, 20, 2, ...], tanh, latente lineal, BCE, Adam 10^{-3}
Adam $\beta_1, \beta_2, \epsilon$	0.9, 0.999, 10^{-8}
Batch / Init	full-batch (32) / Xavier
Epocas	6000
Seeds	[0, 1, 2, 3, 42]
Metrica	max pixel-error (media $\pm$ std)
Criterio conv	1a epoca con $\max \leq 1$

Pregunta: ¿que ingredientes permiten cruzar ≤ 1 pixel, y de forma *robusta* entre seeds?

Variamos **una** pieza a la vez:

1. perdida (BCE / MSE)
2. activacion oculta (tanh / relu / logistica)
3. arquitectura (profundidad / ancho)

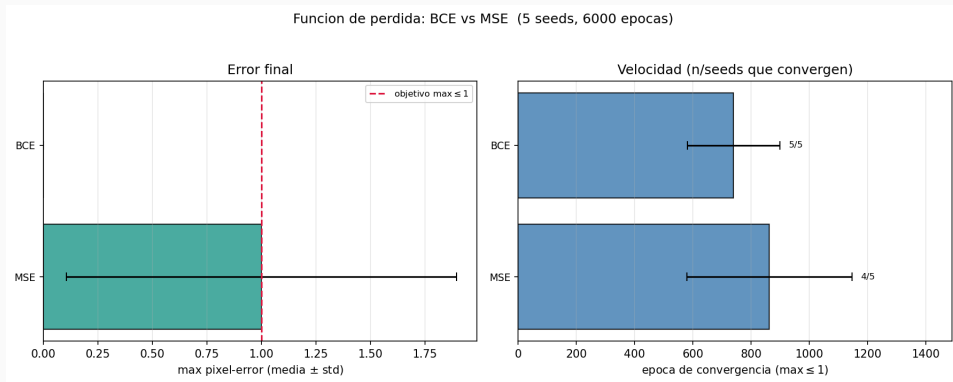
(optimizador $\times$ lr: experimento aparte.)

Eje 1 — Funcion de perdida: parametros

Parametro	Valor
Datos	<code>font.h</code> (32×35)
Arquitectura	[35, 60, 40, 20, 2, 20, 40, 60, 35]
Activacion	tanh / latente lineal / logistica
Optimizador	Adam, $lr = 10^{-3}$
Adam $\beta_1, \beta_2, \epsilon$	0.9, 0.999, 10^{-8}
Batch / Init	full-batch (32) / Xavier
Epocas	6000
Seeds	[0, 1, 2, 3, 42]
<i>Variable</i>	perdida (BCE / MSE)

Pregunta: ¿que funcion de perdida permite alcanzar ≤ 1 pixel de error de forma robusta entre seeds?

Eje 1 — Funcion de perdida: BCE vs MSE



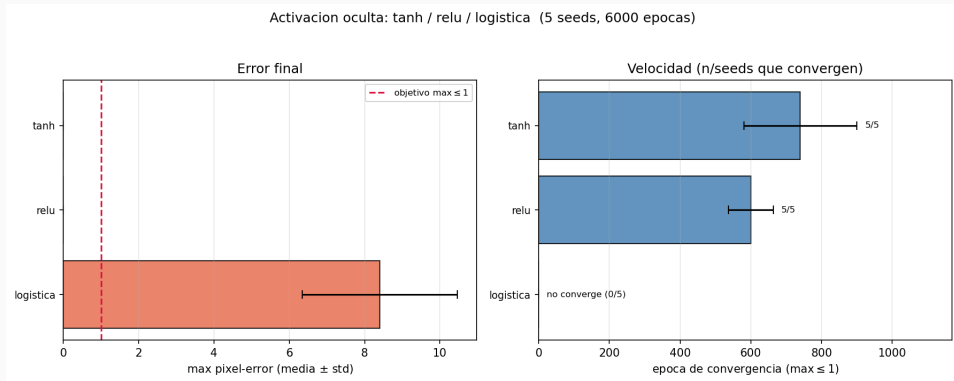
BCE 5/5 vs MSE *borderline* (4/5). Fijamos BCE.

Eje 2 — Activacion oculta: parametros

Parametro	Valor
Datos	font.h (32×35)
Arquitectura	[35, 60, 40, 20, 2, 20, 40, 60, 35]
Perdida	BCE (fijado en Eje 1)
Optimizador	Adam, $lr = 10^{-3}$
Adam $\beta_1, \beta_2, \epsilon$	0.9, 0.999, 10^{-8}
Batch / Init	full-batch (32) / Xavier
Epocas	6000
Seeds	[0, 1, 2, 3, 42]
<i>Variable</i>	activacion oculta (tanh / relu / logistica)

Pregunta: ¿que activacion oculta permite alcanzar ≤ 1 pixel de error de forma robusta entre seeds?

Eje 2 — Activacion oculta: tanh / relu / logistica

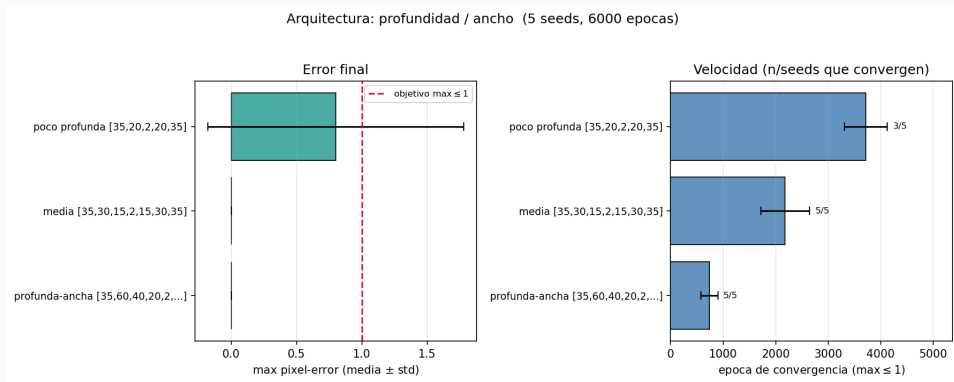


Eje 3 — Arquitectura: parametros

Parametro	Valor
Datos	<code>font.h</code> (32×35)
Activacion	$\tanh$ (fijado en Eje 2)
Perdida	BCE (fijado en Eje 1)
Optimizador	Adam, $\text{lr} = 10^{-3}$
Adam $\beta_1, \beta_2, \epsilon$	0.9, 0.999, 10^{-8}
Batch / Init	full-batch (32) / Xavier
Epocas	6000
Seeds	[0, 1, 2, 3, 42]
<i>Variable</i>	arquitectura (profundidad / ancho)

Pregunta: ¿que arquitectura permite alcanzar ≤ 1 pixel de error de forma robusta entre seeds?

Eje 3 — Arquitectura: profundidad / ancho



poco profunda inestable (3/5); mas profundidad/ancho $\Rightarrow$ confiable. **Fijamos la profunda-ancha** (5/5).

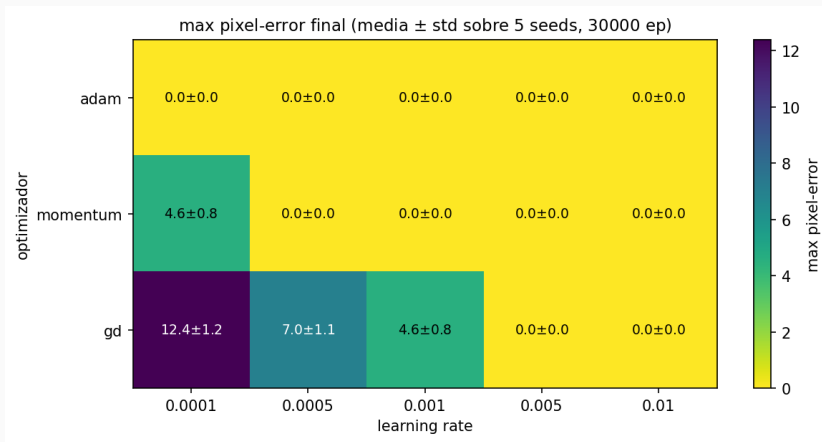
Eje 4 — Optimizador $\times$ learning rate: parametros

Parametro	Valor
Datos	font.h (32×35)
Config base	profunda-ancha, tanh, BCE
Batch / Init	full-batch (32) / Xavier
<i>Variable</i>	optimizador $\times$ lr
optimizador	GD / Momentum(0.9) / Adam
Adam $\beta_1, \beta_2, \epsilon$	0.9, 0.999, 10^{-8}
lr	10^{-4} a 10^{-2} (5 valores)
Protocolo	5 seeds $\times$ 30000 ep

Pregunta:

- ¿el lr optimo *depende* del optimizador?, ¿es Adam robusto al lr?

Eje 4 — Optimizador $\times$ learning rate (5 seeds $\times$ 30000 ep)



Adam: max= 0 en *todo* el rango de lr (robusto). GD y Momentum solo con lr altos. **Fijamos Adam.**

Resultados

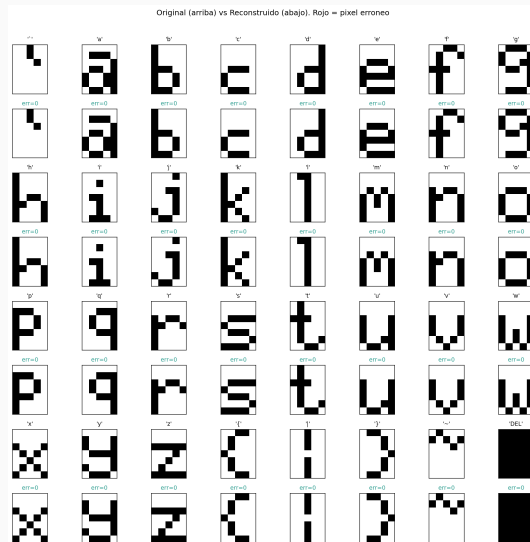
Objetivo ≤ 1 pixel: CUMPLIDO

Configuración del modelo final

Arquitectura	[35, 60, 40, 20, 2, 20, 40, 60, 35]
Activaciones	tanh / latente lineal / logistica
Perdida / Opt	BCE / Adam ($lr = 10^{-3}$)
Adam $\beta_1, \beta_2, \epsilon$	0.9, 0.999, 10^{-8}
Init / Epocas	Xavier / 30000
Early stopping	patience 4000, $\min\Delta = 10^{-7}$
Batch / Seed	full-batch (32) / 0

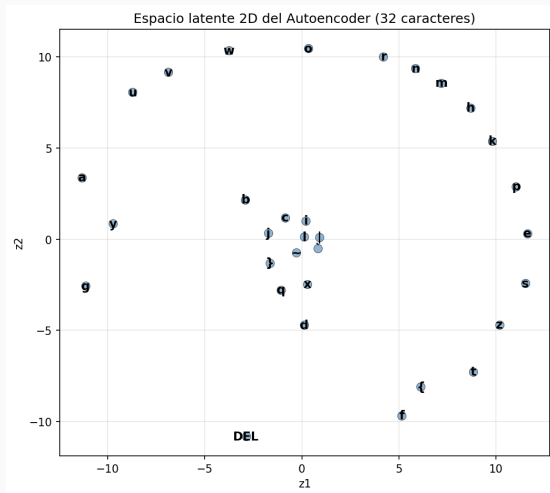
Metrica	Valor
max pixel-error	0
letras exactas	32/32
BCE final (mejor ep)	1.74×10^{-6}

Cumplido con margen ($\max = 0$). Las 5 seeds llegan a $\max = 0$; model.npz reproduce el resultado.



Original (arriba) vs reconstruccion (abajo); rojo = pixel

Espacio latente 2D del AE

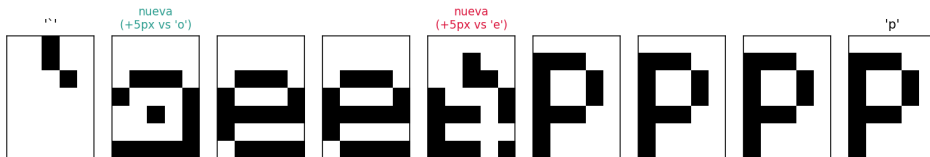


Latente 2D del AE no lineal (32 letras).

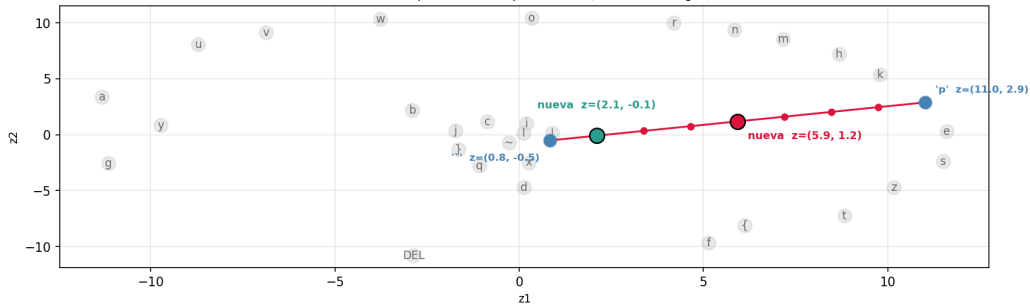
- El AE *no lineal* ubica las 32 letras en posiciones bien separadas del plano 2D.
- Mismo cuello 2D que el PCA, pero aca reconstruye **perfecto** (vs 27 % de varianza del PCA lineal).

Generar una letra nueva (interpolacion)

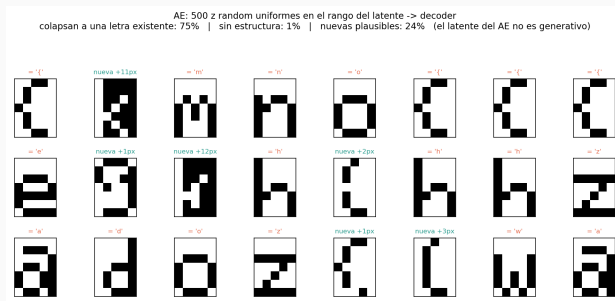
Generacion de letra nueva: interpolacion '' -> 'p' | punto medio a 5px de la letra mas cercana ('e') -> no pertenece al set



Recta de interpolacion en el espacio latente (las 32 letras en gris)



El latente del AE no es generativo



- Muestreamos 500 puntos *z* *al azar* en el rango del latente y decodificamos.
- **75 %** colapsa a una letra existente.
- **25 %** son estructuras que podrian considerarse nuevas letras.

Denoising Autoencoder

Modelos de ruido

- **salt&pepper**: invierte cada pixel con prob. p .
- **gaussian**: $x + \mathcal{N}(0, \sigma^2)$, clip a $[0, 1]$.
- **masking**: pone a 0 una fraccion de pixeles.

A tener en cuenta

- Entrenamos **solo con salt&pepper**.
- El ruido cambia en cada epoca:
online augmentation.

Parametro	Valor
Datos	font.h (32 × 35)
Augmentation	online
Arquitectura	[35, 60, 40, 20, 2, 20, 40, 60, 35]
Activaciones	tanh (oculta), logistica (salida) cuello <i>lineal</i> (identidad)
Loss / Opt	BCE / Adam 10^{-3}
Epocas / Batch / Init	12000 / full-batch / Xavier
Ruido train	salt&pepper $p = 0.10$
Seeds	[0, 1, 2, 3, 42]

Objetivo

- Con una **entrada ruidosa**, el modelo debe reconstruir el **target limpio**.

¿Por qué online?

- Para que el DAE no memorice un patron fijo de ruido.

Estudio 1 — arquitectura × cuello: parametros

Parametro	Valor
Datos	font.h (32 × 35)
Entrenamiento	salt&pepper online $p=0.10$, target limpio
Activaciones	tanh (oculta), identidad (cuello)
Loss / Opt	BCE / Adam 10^{-3}
Batch / Init	full-batch (32) / Xavier
Epocas	12000
Seeds	[0, 1, 2, 3, 42]
Metrica	pixel-error medio a $p=0.10$ + exactas/32 en limpio

Variables: **arquitectura** × **cuello**.

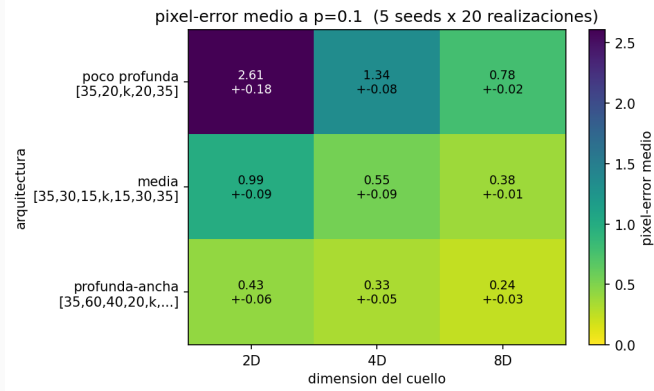
Arquitecturas (mismas que Eje 3 del AE):

- poco profunda [35, 20, k , 20, 35]
- media [35, 30, 15, k , 15, 30, 35]
- profunda-ancha [35, 60, 40, 20, k , ...]

Cuello $k \in \{2, 4, 8\}$.

Pregunta: ¿que combinacion minimiza el error de denoising sin perder reconstruccion?

Estudio 1 — arquitectura × cuello: resultados



Cada celda: píxeles mal reconstruidos por letra (promedio), alimentando el DAE con letras **ruidosas** (salt&pepper $p=0.10$) y midiendo el error contra la letra **limpia**. Promedio sobre 20 reps de ruido $\times$ 5 seeds.

Estudio 2 — nivel de ruido en train: parametros

Parametro	Valor
Datos	font.h (32×35)
Config base	DAE profunda-ancha, cuello 8D (Estudio 1), tanh, BCE, Adam 10^{-3}
Entrenamiento	salt&pepper online, target limpio
Batch / Init	full-batch (32) / Xavier
Epocas	12000
Seeds	[0, 1, 2, 3, 42]
Metrica	pixel-error medio (32 letras $\times$ 20 realizaciones)

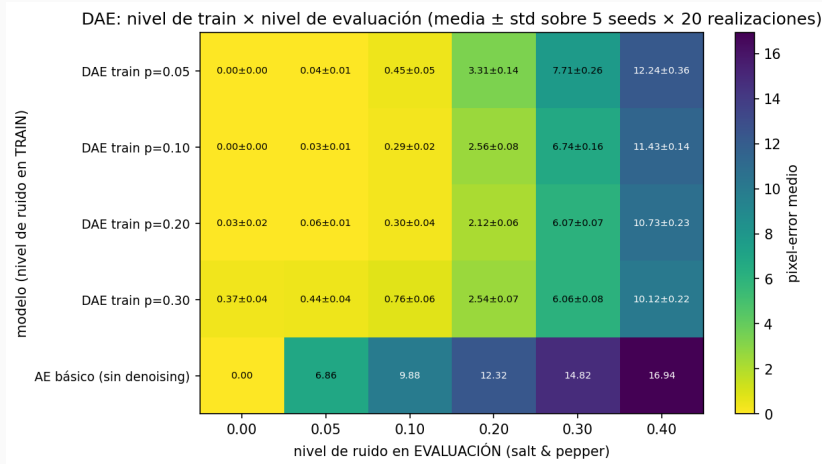
Variable: **nivel de ruido en train**

$p \in \{0.05, 0.10, 0.20, 0.30\}$ (un DAE por nivel).

Pregunta: ¿cuanto ruido conviene en train para generalizar a distintos niveles?

Eval: barriendo el ruido $0 \rightarrow 0.40$; **media** $\pm$ **std** sobre las 5 seeds.

Estudio 2 — ¿Cuanto ruido conviene en train?



Sweet spot $p \approx 0.10\text{--}0.20$: $p = 0.05$ generaliza mal a ruido fuerte; $p = 0.30$ ya degrada el limpio.

Estudio 3 — activacion del cuello: parametros

Parametro	Valor
Datos	font.h (32 × 35)
Config base	DAE profunda-ancha, cuello 8D (Estudios 1–2), tanh, BCE, Adam 10^{-3}
Entrenamiento	salt&pepper online $p=0.10$, target limpio
Batch / Init	full-batch (32) / Xavier
Epocas	12000
Seeds	[0, 1, 2, 3, 42]
Metrica	pixel-error medio (32 letras × 20 realizaciones)

Variable: **activacion del cuello**

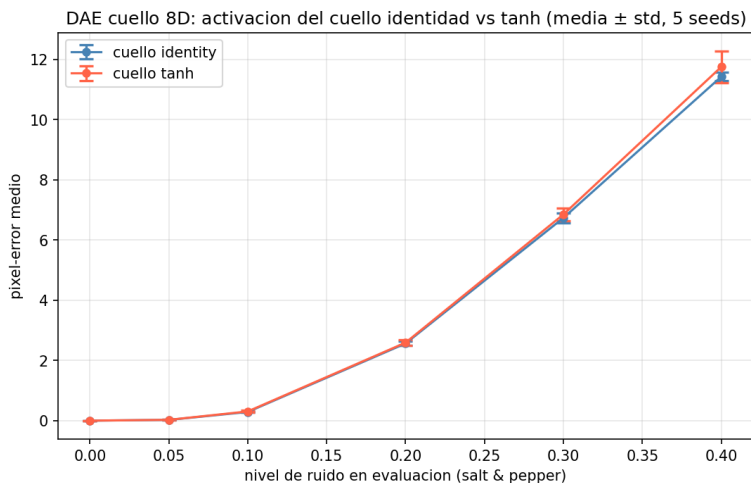
$\in \{\text{identidad, tanh}\}$.

Pregunta: ¿la activacion del latente cambia el denoising? (en 1a el cuello era lineal).

Eval: barriendo el ruido $0 \rightarrow 0.40$; **media** $\pm$ **std** sobre las 5 seeds.

Estudio 3 — ¿Importa la activacion del cuello?

Varia: activacion del cuello, identidad vs tanh (resto fijo: ver parametros). 5 seeds.

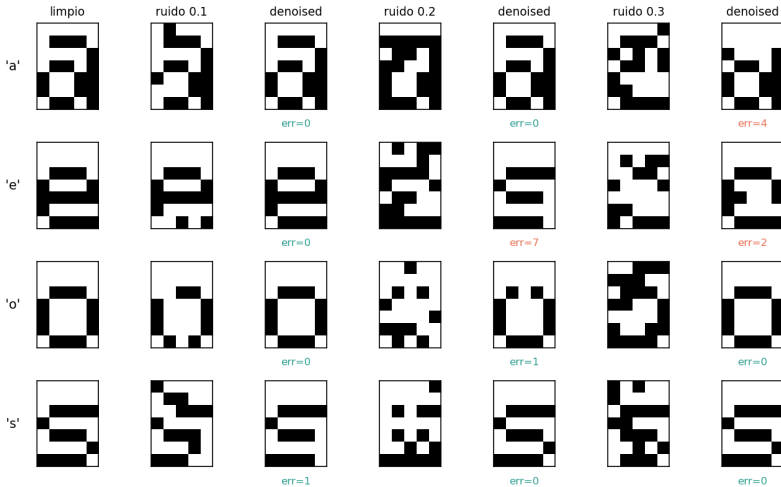


Empate: ambas reconstruyen el limpio (32/32) y dan el mismo denoising dentro del std. Identidad es igual o

Capacidad de denoising (cualitativo)

DAE final: cuello 8D, entrenado con salt&pepper $p = 0.10$.

Denoising AE — ruido 'salt_pepper': limpio / ruidoso / reconstruido



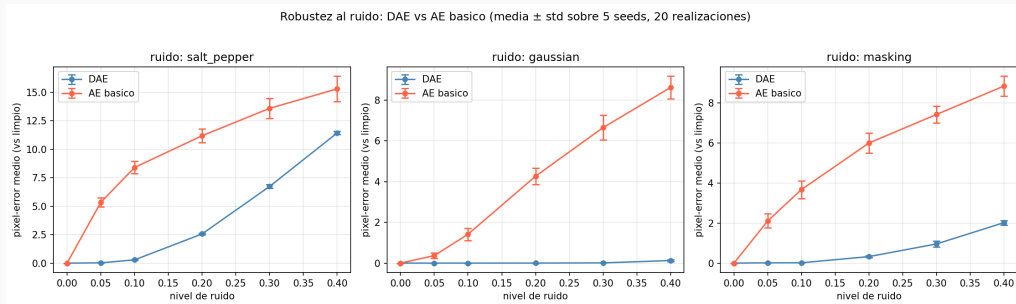
Parametro	Valor
Datos	font.h (32×35)
DAE	cuello 8D lineal, train salt&pepper $p = 0.10$ (online)
AE basico	cuello 2D lineal, <i>sin</i> ruido (input=target)
Loss / Opt	BCE / Adam 10^{-3}
Epocas	DAE 12000 / basico 30000
Seeds	[0, 1, 2, 3, 42]
Metrica	pixel-error medio (32 letras $\times$ 20 realizaciones)

Comparacion: **3 tipos de ruido**

(salt&pepper, gaussian, masking),
barriendo el nivel $0 \rightarrow 0.40$.

Pregunta: ¿el denoising es un efecto
aprendido?

Robustez al ruido: DAE vs AE basico



El DAE queda **muy por debajo** en error en los 3 ruidos, incluso en *gaussian* y *masking* que **no vio en train**.

Conclusiones

- Un AE no lineal con cuello **2D** memoriza las 32 letras con **0 píxeles de error**, muy por encima del baseline PCA 2D.
- **Receta robusta**: BCE + tanh + Adam funciona bien y es insensible al lr.
- El **DAE** limpia salt&pepper hasta $p \approx 0.2-0.3$ y generaliza a ruidos no vistos en train (gaussian, masking).
- Cuello **8D lineal** es óptimo para denoising; el nivel de ruido de train tiene un sweet spot en $p \approx 0.10-0.20$.
- **Límite del AE**: el espacio latente **no tiene estructura**; muestrear puntos arbitrarios no genera letras válidas. $\Rightarrow$ necesitamos un espacio latente organizado.

Autoencoder Variacional (VAE)

Objetivo: construir un espacio latente *organizado* del que se pueda **samplear** para generar imagenes nuevas — lo que el AE basico no permite.

Dataset: emojis 16×16 en escala de grises, 16 clases con 60 variaciones cada una (960 imagenes).

Plan: primero *barremos hiperparametros* y luego corremos un VAE final con la mejor configuracion encontrada.

El dataset: emojis 16×16



16 clases (caras, corazon, estrella, sol, nube, luna, fuego, flor, pulgar), 60 variaciones por clase — 960 imagenes en escala de grises.

Estudio 1 — dimension del latente: parametros

Parametro	Valor
Datos	emojis 16×16 (960)
Arquitectura	encoder/decoder simetricos
Activaciones	ReLU (ocultas) / logistica (salida) / identidad (latente $\mu, \log \sigma^2$)
Loss / Opt	BCE + KL / Adam 10^{-3}
β / Batch	1.0 / 64
Epocas	400
Seeds	[0, 1, 2, 3, 42]
Metrica	rec MSE + calidad visual muestras $z \sim \mathcal{N}(0, I)$

Variable: **dim latente** $\in \{2, 8, 16\}$.

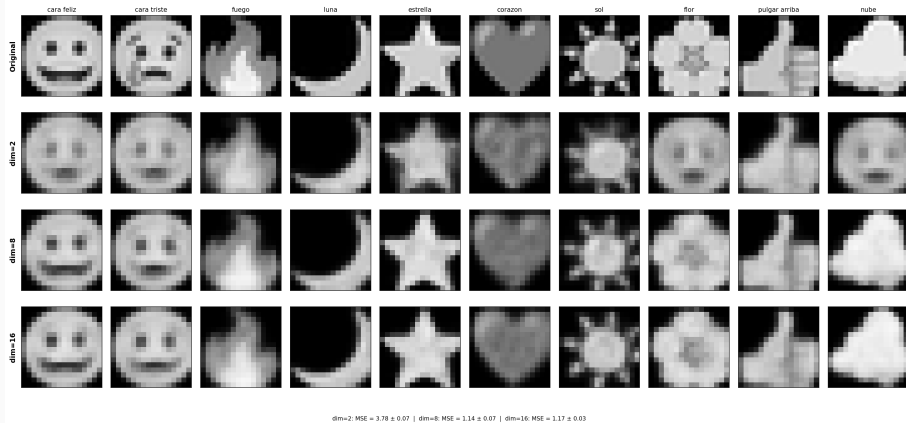
Pregunta: ¿cuantas dimensiones necesita el espacio latente?

2D permite el manifold y la visualizacion directa; mas dims pueden mejorar la reconstruccion a costa de perder interpretabilidad.

Estudio 1 — reconstrucción por dim latente

Varia: dim latente $\in \{2, 8, 16\}$ (resto fijo: seed=42).

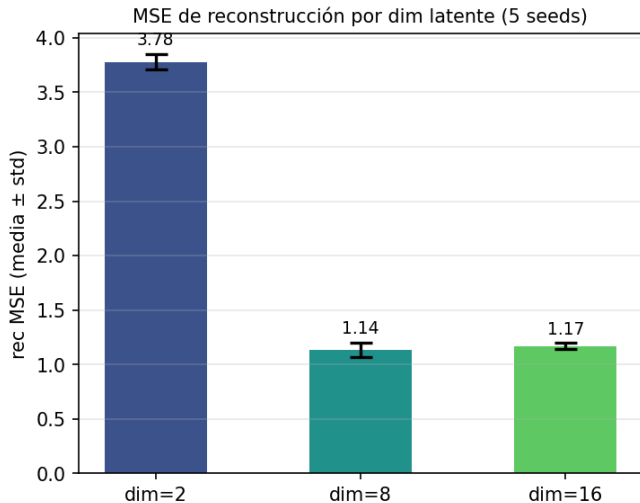
Reconstrucción por dimensión latente ($z = \mu(x)$, seed=42)



Reconstrucción con $z = \mu(x)$ (determinístico). Mayor dim $\Rightarrow$ más detalle.

Estudio 1 — MSE por dim latente

Varia: dim latente $\in \{2, 8, 16\}$ (resto fijo: 5 seeds [0,1,2,3,42], 1000 epoch).



Estudio 2 — efecto de β : parametros

Parametro	Valor
Datos	emojis 16×16 (960)
Arquitectura	encoder/decoder, latente 2D
Activaciones	ReLU (ocultas) / logistica (salida) / identidad (latente $\mu, \log \sigma^2$)
Loss / Opt	BCE + $\beta \cdot \text{KL}$ / Adam 10^{-3}
Batch	64
Epocas	400
Seeds	[0, 1, 2, 3, 42]
Metrica	scatter (seed rep.), rec, KL

Variable: $\beta \in \{0.1, 1.0, 4.0\}$.

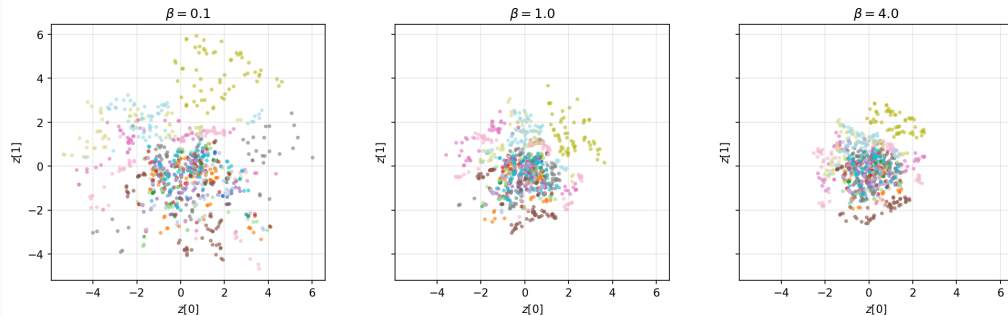
Pregunta: ¿que β organiza el latente sin sacrificar reconstruccion?

β bajo $\Rightarrow$ latente libre, mala generacion. β alto
 $\Rightarrow$ clusters colapsados, rec peor.

Estudio 2 — espacio latente por β

Varia: $\beta \in \{0.1, 1.0, 4.0\}$ (resto fijo: ver parametros). Seed representativa.

Estudio 2 — espacio latente por β (seed representativa, ejes comunes)

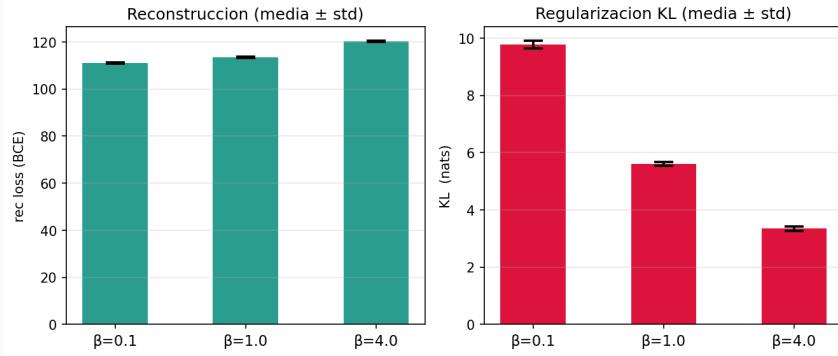


Mismos ejes en los 3 paneles: al subir β la nube se **contrae hacia el origen** (KL menor $\Rightarrow$ mas cerca del prior $\mathcal{N}(0, I)$). $\beta = 0.1$ ocupa todo el rango; $\beta = 4.0$ colapsa al centro.

Estudio 2 — tradeoff rec/KL por β

Varia: $\beta \in \{0.1, 1.0, 4.0\}$ (resto fijo: ver parametros). 5 seeds [0,1,2,3,42].

Estudio 2 — tradeoff rec/KL por β (5 seeds)



$\beta = 0.1$: mejor rec pero KL alto (latente libre). $\beta = 4.0$: rec deteriorada (KL= 3.35, clusters colapsados).

$\beta = 1.0$: equilibrio elegido (rec= 113.5, KL= 5.61).

Estudio 3 — reconstruccion BCE vs MSE: parametros

Parametro	Valor
Datos	emojis 16×16 (960)
Arquitectura	[256, 64]/[64, 256], latente 2D
Salida	logistica (pixeles en (0, 1))
Opt / Epocas	Adam 10^{-3} / 600
Seeds	[42, 0, 1]
Metrica	rec-MSE (comun), KL, generacion

Variable: **recon_loss** $\in \{\text{BCE}, \text{MSE}\}$.

Pregunta: ¿que loss conviene en **2D** (la corrida final), mirando reconstruccion y *generacion*?

Lo evaluamos en **2D** (no 16D): es la dim que usamos de verdad, y nos importa la *consecuencia generativa*, no solo el detalle de reconstruccion.

BCE = verosimilitud Bernoulli del pixel:

gradiente limpio ($\hat{x} - x$) con salida logistica.

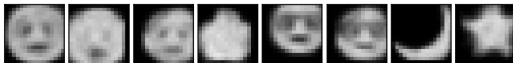
MSE+logistica: el gradiente se *desvanece* en pixeles saturados $\Rightarrow$ tiende a la media (gris).

Estudio 3 — BCE vs MSE: resultados

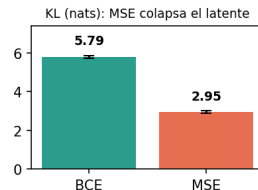
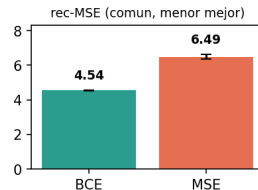
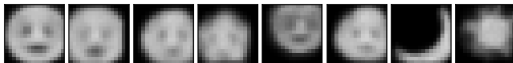
Varia: $\text{recon_loss} \in \{\text{BCE}, \text{MSE}\}$ en latente 2D (resto fijo: ver parametros). 3 seeds [42,0,1].

Estudio 3 (2D) --- BCE vs MSE: con MSE el latente colapsa y la generacion se va a gris
Generacion $z \sim N(0, I)$ (latente 2D, mismos z)

BCE



MSE



En 2D, **MSE colapsa el latente** (KL 2.95 vs 5.79): al ser el termino de reconstruccion mas chico, el KL pesa relativamente mas y el encoder manda los codigos al centro $\Rightarrow$ menos info al decoder $\Rightarrow$ generacion mas

Estudio 4 — capacidad de la arquitectura: parametros

Parametro	Valor
Datos	emojis 16×16 (960)
Loss / Opt	BCE / Adam 10^{-3}
Latente / Batch	2D / 64
Epocas	400
Seeds	[42, 0, 1]
Metrica	rec-MSE, rec_BCE, KL, loss = rec + β KL, n params

Variable: **ancho/profundidad** del
encoder/decoder:

angosta [128]/[128] (67k)

base [256, 64]/[64, 256] (165k)

ancha [512, 128]/[128, 512] (396k)

profunda [256, 128, 64]/... (215k)

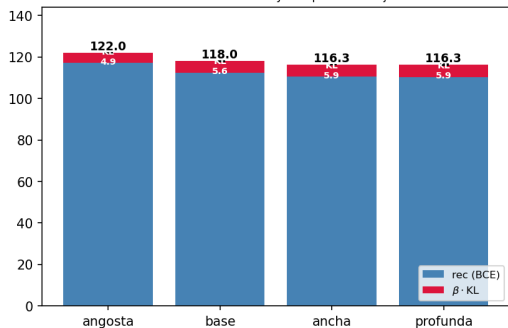
Pregunta: ¿cuanta capacidad fuera del
cuello 2D conviene?

Estudio 4 — arquitectura: resultados

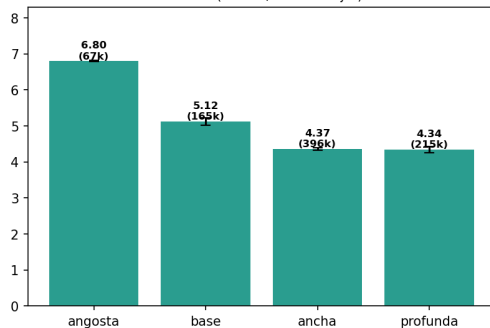
Varia: ancho/profundidad encoder/decoder (resto fijo: ver parametros). 3 seeds [42,0,1].

Estudio 4 (2D) --- capacidad: loss completa vs rec-MSE (etiqueta = nº params)

Loss completa = $\text{rec}(\text{BCE}) + \beta \cdot \text{KL}$ (menor mejor):
casi no cambia --- el KL sube y compensa la mejora en rec



rec-MSE (comun, menor mejor)



Por **rec-MSE** la capacidad parece importar mucho ($6.80 \rightarrow 5.12 \rightarrow 4.34$), con **rendimientos decrecientes**; la **profunda** iguala a la ancha (4.34 vs 4.37) con $\sim$ mitad de params. Pero por la **loss completa** ($\text{rec} + \beta \text{KL}$) las 4 quedan **casi iguales** ($122 \rightarrow 116$): al subir la capacidad mejora la rec pero **sube el KL** ($4.9 \rightarrow 5.9$), que *compensa* ^{50/60}

Estudio 5 — KL warmup: parametros

Parametro	Valor
Datos	emojis 16×16 (960)
Arquitectura	encoder/decoder, latente 2D
Loss / Opt	BCE + KL / Adam 10^{-3}
β final / Batch	1.0 / 64
Epocas	400
Seeds	[0, 1, 2, 3, 42]
Metrica	curvas rec y KL por epoca

Variable: **warmup** $\in \{0, 100\}$ epocas.

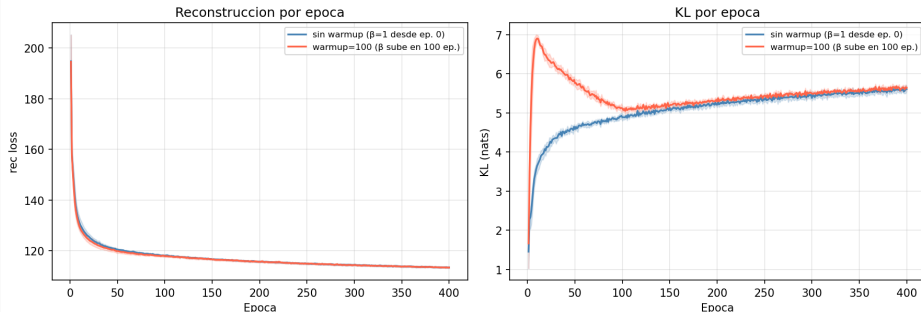
Con warmup, β sube *linealmente* de 0 a 1 en las primeras N epocas; sin warmup, $\beta = 1$ desde el inicio.

Pregunta: ¿el annealing del KL estabiliza el entrenamiento y mejora la convergencia?

Estudio 5 — efecto del KL warmup

Varia: warmup $\in \{0, 100\}$ épocas (resto fijo: ver parametros). 5 seeds [0,1,2,3,42].

Estudio 4 — KL warmup: media $\pm$ std, 5 seeds, 400 épocas



Ambas convergen practicamente igual ($\text{rec} \approx 113.4\text{--}113.5$, $\text{KL} \approx 5.61\text{--}5.64$). **Warmup no es determinante** $\Rightarrow$ usamos $\beta = 1$ desde ep. 0.

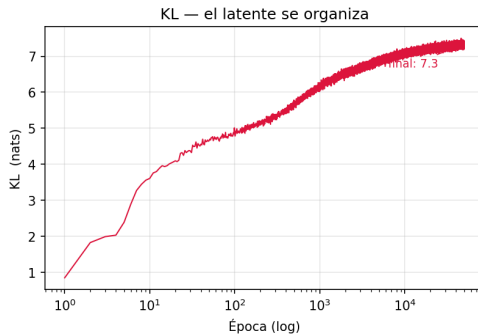
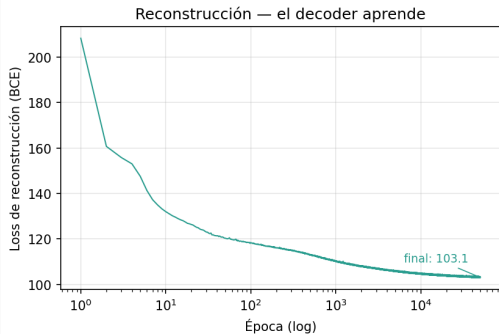
Parametro	Valor
Datos	emojis 16×16 gris 16 clases, 60/clase (960)
Encoder	$256 \rightarrow 256 \rightarrow 64 \rightarrow (\mu, \log \sigma^2) \in \mathbb{R}^2$
Decoder	$\mathbb{R}^2 \rightarrow 64 \rightarrow 256 \rightarrow 256$
Activaciones	ReLU (ocultas) / logistica (salida) identidad $(\mu, \log \sigma^2)$
Loss / Opt	BCE + $\beta \cdot \text{KL}$ / Adam 10^{-3}
β / Batch	1.0 / 64
Epocas / Seed	50.000 / 42

De los experimentos: latente 2D
(manifold y visualizacion directa), **BCE**,
 $\beta = 1.0$ (sweet spot rec/KL),
arquitectura base y **sin KL warmup**.

Con esa configuracion entrenamos una corrida
larga (50.000 epocas) y analizamos el latente, la
reconstruccion, el manifold generativo y la
generacion de muestras nuevas.

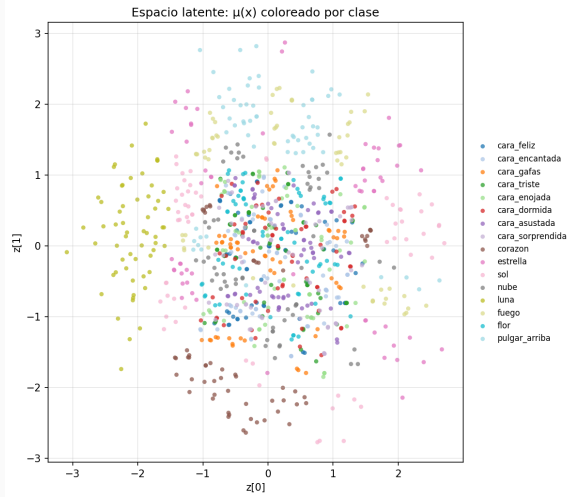
Curvas de aprendizaje

Curvas de aprendizaje del VAE (50.000 épocas)

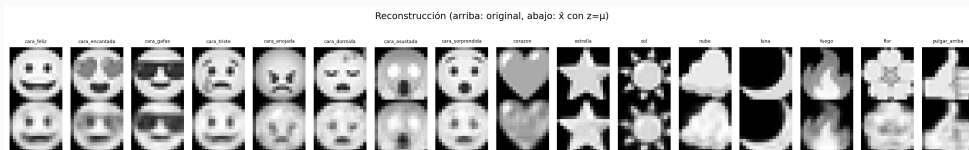


Rec baja: el decoder aprende a reconstruir. **KL** sube y se estabiliza: el encoder organiza el latente.

El latente se organiza

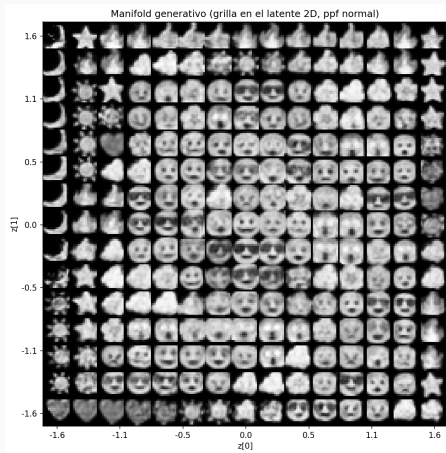


Cada punto es $\mu(x)$. El KL forzo el latente hacia $\mathcal{N}(0, I)$: clusters por clase cerca del origen, **continuos y separados**.



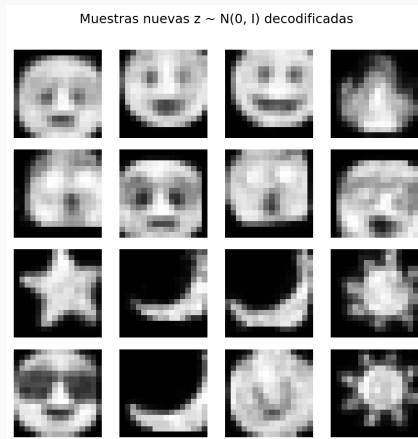
Arriba: original; abajo: $\hat{x}$ con $z = \mu$ (determinístico). Fiel en las 16 clases.

Manifold generativo



El decoder genera emojis plausibles en distintas regiones del plano, mostrando un **manifold continuo**: puntos cercanos producen imágenes parecidas.

Generacion: nuevas muestras



$z \sim \mathcal{N}(0, I)$ decodificados: el VAE genera imágenes **nuevas y reconocibles** del dataset.

Interpolacion en el latente



Interpolacion fija entre `cara_feliz` y `corazon`: la transicion se vuelve mas visible entre clases visualmente distintas.

Conclusiones EJ2

- El VAE impone $p(z) = \mathcal{N}(0, I)$ via KL $\Rightarrow$ latente **continuo y organizado**: clusters por clase, interpolacion suave.
- **Manifold 2D**: todo punto del latente genera un emoji valido; la grilla 15×15 lo confirma visualmente.
- **Generacion real**: $z \sim \mathcal{N}(0, I)$ decodificado produce imagenes reconocibles del dataset.
- **Dim latente = trade-off**: para reconstruccion pura conviene **8D** (la rec satura ahi: 16D ya no aporta), pero elegimos **2D** a proposito — es lo unico que permite graficar el *manifold* y el latente *directamente*, asumiendo el costo en detalle.
- $\beta = 1.0$: sweet spot rec/KL; KL warmup no es determinante en este caso.